ИТМО

Алгоритмы и структуры данных

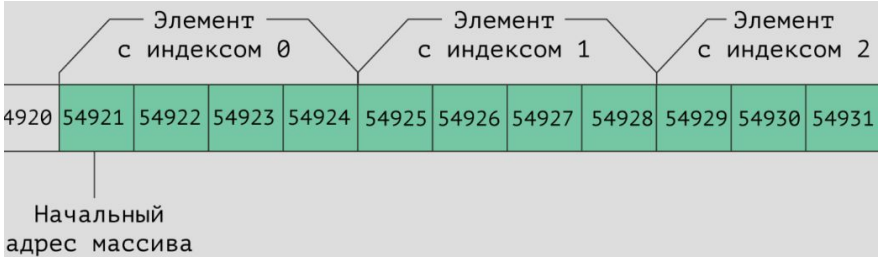
Лекция 6

Динамические структуры данных. Массив, списки.

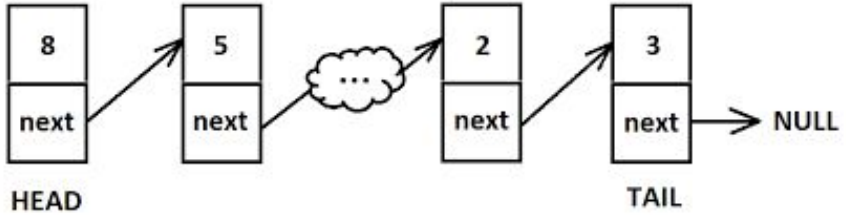
Пример задач

ИТМО

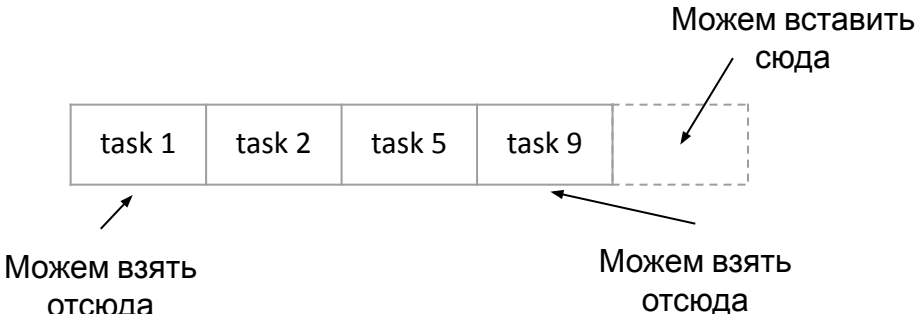
Массив



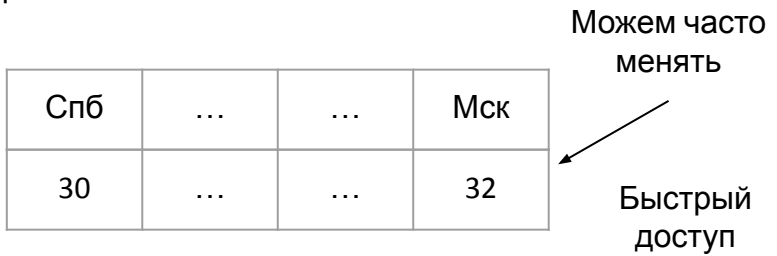
Список



Задача 1.
Реализовать очередь задач. Задачи поступают в конец и могут изыматься с двух концов



Задача 2.
Хранение и доступ температуры по регионам. Нужно хранить температуру по всем регионам и уметь быстро ее отдавать



Асимптотика и размер

ИТМО

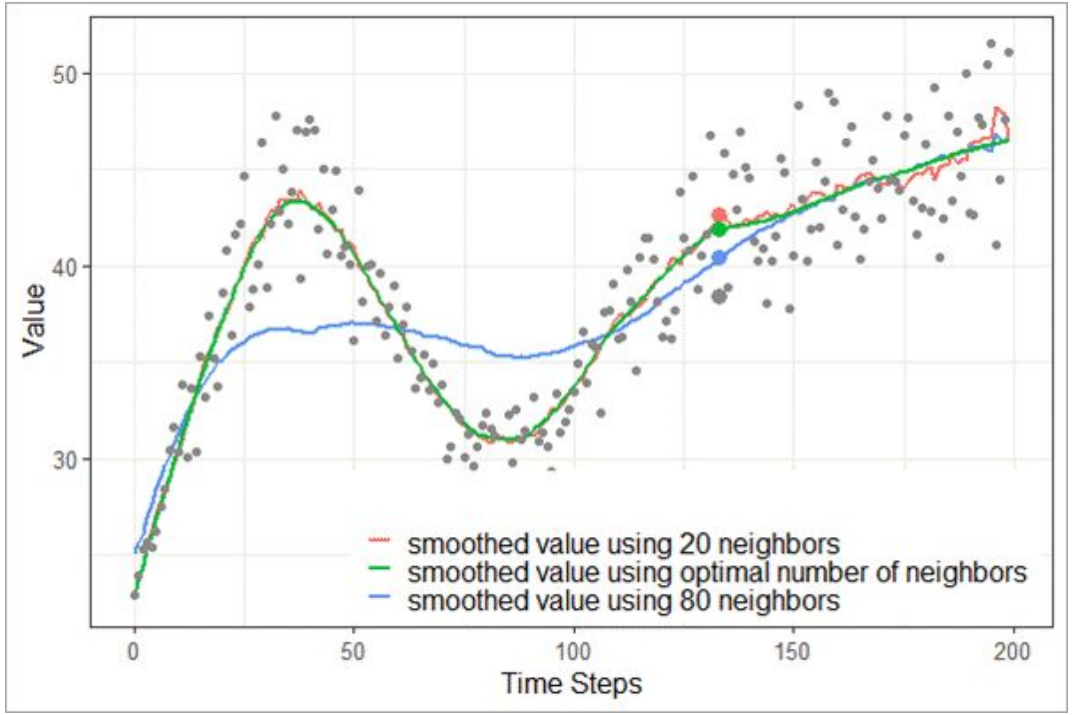
Структура данных	Array	LinkedList	Double LinkedList
index[i]	O(1)	O(N)	O(N)
index[0], index[-1]	O(1)	O(1)	O(1)
append	O(N), O*(1)	O(1)	O(1)
set(index, value)	O(1)	O(N)	O(N)
pop(0), pop(-1)	O(N), O*(1)	O(1), O(N)	O(1)
pop(i)	O(N)	O(N)	O(N)
find(value)	O(N)	O(N)	O(N)
size	N * sizeof(el) + 12 bytes	N * 16 + 8 bytes	N * 24 + 8 bytes

Пока для справки

Начнем (и закончим) с задачи

ИТМО

Сглаживание временного ряда средним значением



Вход - файл с числами разделенные пробелом

Задача - сгладить ряд с заданным размером окна

Выход - файл с числами разделенные пробелом

Чтение данных

ИТМО

Начнем с реализации функции чтения данных

```
int8_t* read_array(char*
filename)
```

Нам нужно:

1. Определить логические шаги (псевдокод)
2. Задуматься
3. Реализовать логические шаги
4. Задуматься
5. Придумать тесты
6. Задуматься
7. Реализовать тесты
8. Задуматься

1. Подумать об ограничениях работы с файлом и вытекающих последствий
2. Подумать о рисках работы с динамической памяти

Псевдокод для чтения файла

ИТМО

```
int8_t* read_array(char* filename) {
    // Создать массив, в который будем читать данные из файла

    // Открыть файл

    // Инициализировать переменные для хранения строки-числа и текущего символа
    // Прочитать первый символ
    // Пока символ не EOF (не конец файла)
        // Если символ - это пробел
            // Добавить число в массив и увеличить размер массива
            // Обнулить строку число
        // Иначе
            // Добавить символ в строку-число
        // Прочитать следующий символ

    // Закрыть файл
}
```

Остановиться и подумать!

Работа с файлам

```
int8_t* read_array(char* filename) {
    int arr_size = 0;  // Почему 0 и arr_size + 1?
    int8_t* arr = malloc((arr_size + 1) * sizeof(int8_t));
    if (arr == NULL) RETURN_ERROR("Не выделить память под массив ", NULL);

    FILE *file = fopen(filename, "r");
    if (!file) RETURN_ERROR("Не удалось открыть файл ", NULL);

    char digit_str[5] = "";  // Почему 5?
    int symbol_code = fgetc(file);  // Чтение следующего
    while (symbol_code != EOF) {
        // ...

    fclose(file); // Что забыли перед fclose?
}
```

Теперь надо подумать
что будет вот тут

База без важных моментов

ИТМО

```
while (symbol_code != EOF) {
    char symbol = (char)symbol_code;
    if (symbol == ' ') {
        char *end;
        // strtol - str -> int. В каком случае тут можем словить ошибку?
        arr[arr_size] = (int8_t)strtol(digit_str, &end, 10);
        arr_size++;
        // Что нужно сделать после установки значения?
        digit_str[0] = '\0';
    } else {
        // Какую ошибку можем словить?
        digit_str[len] = symbol;
        digit_str[len + 1] = '\0';

    }
    symbol_code = fgetc(file);
}
```

Я этот кусок вижу так

ИТМО

```
while (symbol_code != EOF) {
    char symbol = (char)symbol_code;
    if (symbol == ' ') {
        if (strlen(digit_str) > 0) {
            char *end;
            arr[arr_size] = (int8_t)strtol(digit_str, &end, 10);
            arr_size++;
            arr = realloc(arr, (arr_size + 1) * sizeof(int8_t));
            if (arr == NULL) {
                free(arr);
                RETURN_ERROR("Не удалось релоцировать память", NULL);
            }
            digit_str[0] = '\0';
        }
    } else {
        // Тут проверить что в файле нет больших чисел
        size_t len = strlen(digit_str);
        digit_str[len] = symbol;
        digit_str[len + 1] = '\0';
    }
    symbol_code = fgetc(file);
}
```

```
int8_t* read_array(char* filename) {
    int arr_size = 0;
    int8_t* arr = malloc((arr_size + 1) * sizeof(int8_t));
    if (arr == NULL) RETURN_ERROR("Не выделить память под массив", NULL);
    FILE *file = fopen(filename, "r");
    if (!file) RETURN_ERROR("Не удалось открыть файл", NULL);
    char digit_str[5] = ""; int symbol_code = fgetc(file);
    while (symbol_code != EOF) {
        char symbol = (char)symbol_code;
        if (strlen(digit_str) == 3) {
            if (strlen(digit_str) > 0) {
                char *end;
                arr[arr_size] = (int8_t)strtol(digit_str, &end, 10);
                arr_size++;
                arr = realloc(arr, (arr_size + 1) * sizeof(int8_t));
                if (arr == NULL) …
                digit_str[0] = '\0';
            }
        } else {
            if (strlen(digit_str) >= 5) …
            digit_str[strlen(digit_str)] = symbol;
            digit_str[strlen(digit_str) + 1] = '\0';
        }
        symbol_code = fgetc(file);
    }
    if (strlen(digit_str) > 0) {
        char *end;
        arr[arr_size] = (int8_t)strtol(digit_str, &end, 10);
        arr_size++;
    }

    fclose(file);
    return arr;
```

На всякий случай код, чтобы был пример чтения файлов

Остановиться и подумать!

В чем главные проблемы?

И давайте над тестами подумаем=)

Хорошо, примеры тестов

ИТМО

Функциональные:

1. положительные “1 2 3 4 5”
2. отрицательные “-1 -2 -3 -4- 5”
3. смешанные “-1 1 -2 2 3”
4. нули “0 0 1 0 0”
5. несколько пробелов “1 1 1 1”
6. чужеродные символы “a 1 2 3”
7. числа с ведущими нулями “01 02 03”

```
// У нас функция объявляется так
int8_t* read_array(char* filename)

// Вот так её можно вызвать
int8_t* arr = read_array("test_data.txt");
```

Вопрос: а как узнать размер arr?

Граничные:

1. файл не существует
2. пустой файл “”
3. файл с пробелами “ ”
4. минимальное значение “-128”
5. максимальное значение “127”
6. выход за пределы “-129”
7. выход за пределы “128”
8. предельный размер числа “12345”
9. за пределы размера числа “123456”
10. 1e4 чисел “1 1 1 1 … 1 1 1”
11. 1e7 чисел (тест на утечку памяти)

```
int8_t* read_array(char* filename, size_t* arr_size) {
    int8_t* arr = malloc((*arr_size + 1) * sizeof(int8_t));
    if (arr == NULL) RETURN_ERROR("Не выделить память под массив", NULL);
    FILE *file = fopen(filename, "r");
    if (!file) RETURN_ERROR("Не удалось открыть файл", NULL);
    char digit_str[5] = ""; int symbol_code = fgetc(file);
    while (symbol_code != EOF) {
        char symbol = (char)symbol_code;
        if (strlen(digit_str) == ' ') {
            if (strlen(digit_str) > 0) {
                char *end;
                arr[*arr_size] = (int8_t)strtol(digit_str, &end, 10);
                (*arr_size)++;
                arr = realloc(arr, (*arr_size + 1) * sizeof(int8_t));
                if (arr == NULL) …
                digit_str[0] = '\0';
            }
        } else {
            if (strlen(digit_str) >= 5) …
            digit_str[strlen(digit_str)] = symbol;
            digit_str[strlen(digit_str) + 1] = '\0';
        }
        symbol_code = fgetc(file);
    }
    if (strlen(digit_str) > 0) {
        char *end;
        arr[*arr_size] = (int8_t)strtol(digit_str, &end, 10);
        *arr_size++;
    }

    fclose(file);
    return arr;
```

На всякий случай **правильный** код, чтобы был пример чтения файлов

Дальше функция сглаживания

ИТМО

```
int8_t* smoothV1(const int8_t* arr, const size_t arr_size, size_t* size_out, const
u_int8_t window_size) {
    // Валидация входных данных

    // Выделить память под выходной массив
    // Создать массив для хранения значений в окне

    // Цикл по всем элементам (до size - window_size + 1)
        // Заполнить массив окна

        // Посчитать среднее и добавить его в массив

        // Увеличить массив

    return arr_out;
}
```

```c
int8_t* smoothV1(const int8_t* arr, const size_t arr_size, size_t* size_out, const u_int8_t window_size) {
    if (window_size < 2) RETURN_ERROR("windows size min - 2", NULL);
    if (window_size >= arr_size) RETURN_ERROR("windows size must be less than arr", NULL);

    *size_out = 0;
    int8_t* arr_out = malloc((*size_out + 1) * sizeof(int8_t));
    if (arr_out == NULL) RETURN_ERROR("Ошибка выделения памяти под массив", NULL);

    int8_t window[window_size];
    for (int i = 0; i < arr_size - window_size + 1; i++) {
        for (int j = 0; j < window_size; j++) {
            window[j] = arr[i + j];
        }

        arr_out[*size_out] = meanValue(window, window_size);
        *size_out += 1;
        arr_out = realloc(arr_out, (*size_out + 1) * sizeof(int8_t));
        if (arr_out == NULL) {
            free(arr_out);
            RETURN_ERROR("Не удалось релоцировать память", NULL);
        }
    }
    return arr_out;
}
```

Что не нравится?

Не нравится

ИТМО

1. Напрягает выделять память
2. Напрягает перевыделять память
3. Напрягает отдельно хранить размер массива
4. Долгое заполнение окна при сглаживании

Массив

ИТМО

```c
typedef struct {
 int8_t* data;       // Указатель на блок данных
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;
```

```c
Vector *createVector();
int appendVectorItem(Vector *vector, int8_t* el);
void *getVectorItem(Vector *vector, size_t index);
int setVectorItem(Vector *vector, size_t index, int8_t* value);
int8_t* popVectorItem(Vector *vector, size_t index);
long int findVectorItem(Vector *vector, int8_t* value);
int vectorFree(Vector *vector);
```

```c
Vector* readArrayV2(char* filename) {
    Vector* arr = createVector();
    if (arr == NULL) RETURN_ERROR("Error vector creation", NULL);

    FILE *file = fopen(filename, "r");
    if (!file) RETURN_ERROR("Error open file", NULL);

    char digit_str[5] = "";
    int symbol_code = fgetc(file);  // Чтение следующего
    while (symbol_code != EOF) {
        char symbol = (char)symbol_code;
        if (symbol == ' ') {
            if (strlen(digit_str) > 0) {
                char *end;
                int8_t val = (int8_t)strtol(digit_str, &end, 10);
                if (appendVectorItem(arr, &val) == -1) ...
                digit_str[0] = '\0';
            }
        } else {
            size_t len = strlen(digit_str);
            if (len >= 5) ...
            digit_str[len] = symbol;
            digit_str[len + 1] = '\0';
        }
        symbol_code = fgetc(file);
    }
    if (strlen(digit_str) > 0) {
        char *end;
        int8_t val = (int8_t)strtol(digit_str, &end, 10);
        if (appendVectorItem(arr, &val) == -1) ...
    }
    fclose(file);
    return arr;
```

Так выглядит функция чтения с использованием вектора

Функция сглаживания

```
Vector* smoothV2(Vector* arr, const u_int8_t window_size) {
    if (window_size < 2) RETURN_ERROR("Wrong size", NULL);
    if (window_size >= arr->size) RETURN_ERROR("Wrong size", NULL);

    Vector* arr_out = createVector();
    if (arr_out == NULL) RETURN_ERROR("Error vector creation", NULL);

    int8_t window[window_size];
    for (int i = 0; i < arr->size - window_size + 1; i++) {
        for (int j = 0; j < window_size; j++) {
            window[j] = getVectorItem(arr, i + j);
        }

        int8_t value = meanValue(window, window_size);
        if (appendVectorItem(arr_out, &value) == -1) {
            vectorFree(arr_out);
            RETURN_ERROR("Не удалось релоцировать память", NULL);
        }
    }

    return arr_out;
}
```

Давайте чуть поговорим о функциях работы с массивом

ИТМО

```
typedef struct {
 int8_t* data;       // Указатель на блок данных
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;
```

Как нам создать (create) такую структуру?

Описание логических блоков создания вектора

ИТМО

```
typedef struct {
 int8_t* data;       // Указатель на блок данных
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;


Vector *createVector()
{
    // Выделить память через malloc для вектора и проверить
    // Выделить память под данные (и проверить что память выделилась)
    // Установить остальные атрибуты
}
```

Пример реализации создания вектора

```
Vector *createVector()
{
    Vector *vector = malloc(sizeof(Vector));
    if (!vector) return NULL;

    vector->data = malloc(MIN_SIZE * 1);
    if (!vector->data) {
        free(vector);
        return NULL;
    }

    vector->size = 0;
    vector->capacity = MIN_SIZE;

    return vector;
}
```

```
typedef struct {
 int8_t* data;
 size_t size;
 size_t capacity;
} Vector;
```

Функция добавления нового элемента

ИТМО

```
typedef struct {
 int8_t* data;       // Указатель на блок данных
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;

Vector *appendVectorItem(Vector *vector, int8_t* el) {}
```

Как нам добавить элемент?

Логическое описание добавление нового элемента

```
typedef struct {
 int8_t* data;       // Указатель на блок данных
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;


Vector *appendVectorItem(Vector *vector, int8_t* el)
{
    // Проверить, помещается ли новый элемент в массив
    // Если не помещается, увеличить “вместимость” массива
    // Вставить значение в массив
    // Увеличить размер массива
}
```

Реализация добавления элемента

ИТМО

```
int appendVectorItem(Vector* vector, int8_t* el) {
    if (!vector || !el) return -1;

    if (resize(vector, 1) != 0) return -1;

    vector->data[vector->size] = el;
    vector->size++;

    return 0;
}
```

```
typedef struct {
 int8_t* data;
 size_t size;
 size_t capacity;
} Vector;
```

Реализация удаления элемента

ИТМО

```c
int8_t* popVectorItem(Vector *vector, size_t index)
{
    if (!vector || index >= vector->size) return NULL;

    int8 t* copy = malloc(1);
    if (!copy) return NULL;

    int8_t* item = getVectorItem(vector, index);
    memcpy(copy, item, 1);

    // Сдвигаем элементы влево
    for (size t i = index; i < vector->size - 1; i++) {
        int8_t* src = getVectorItem(vector, i + 1);
        setVectorItem(vector, i, src);
    }

    vector->size--;
    resize(vector, -1);

    return copy;
}
```

```c
typedef struct {
 int8_t* data;
 size_t size;
 size_t capacity;
} Vector;
```

Серьезное ограничение нашей реализации?

ИТМО

```
Vector *createVector()
{
    Vector *vector = malloc(sizeof(Vector));
    if (!vector) return NULL;

    vector->data = malloc(MIN_SIZE * 1);
    if (!vector->data) {
        free(vector);
        return NULL;
    }

    vector->size = 0;
    vector->capacity = MIN_SIZE;

    return vector;
}
```

```
int appendVectorItem(Vector* vector, int8_t* el)
{
    if (!vector || !el) return -1;

    if (resize(vector, 1) != 0) return -1;

    vector->data[vector->size] = el;
    vector->size++;

    return 0;
}
```

Серьезное ограничение нашей реализации?

ИТМО

```
Vector *createVector()
{
    Vector *vector = malloc(sizeof(Vector));
    if (!vector) return NULL;

    vector->data = malloc(MIN_SIZE * 1);
    if (!vector->data) {
        free(vector);
        return NULL;
    }

    vector->size = 0;
    vector->capacity = MIN_SIZE;

    return vector;
}
```

```
int appendVectorItem(Vector* vector, int8_t* el)
{
    if (!vector || !el) return -1;

    if (resize(vector, 1) != 0) return -1;

    vector->data[vector->size] = el;
    vector->size++;

    return 0;
}
```

Можем работать только с одним типом

Как нам преодолеть это ограничение?

void*

`void*` — это **указатель на область памяти, тип которой не определён**.

Он может **указывать на данные любого типа**, но **сам по себе НЕ ЗНАЕТ**, какого именно.

```
int x = 10;
void* ptr = &x;  // ptr — указатель на int, но тип void* (без типа)
```

`void*` хранит **адрес**, как и любой другой указатель (`int*`, `char*`, …)
но **разыменовать (`*ptr`) нельзя**, потому что компилятор **не знает**, сколько байт нужно прочитать.

```
void* p;
*p = 5; // ❌ ошибка — неизвестно, сколько байт
писать
```

Как его использовать

```c
int x = 10;
void* p = &x;          // сохраняем адрес int в void*

// Чтобы использовать значение:
int y = *(int*)p;      // привели обратно к int*
printf("%d\n", y);     // выведет 10

int* arr = malloc(10 * sizeof(int));    // malloc возвращает void*
char* buf = malloc(100);                // и это тоже void*
```

Понимаете к чему я веду?

Как его использовать

ИТМО

```
const size_t elem_size = sizeof(int);
int* arr = malloc(10 * elem_size);


// Установка элемента
int value = 4;
memcpy((char*)arr + 3 * elem_size, &value, elem_size);  // #include <string.h>
```

Для int и для индекса 3 нам нужно сместиться на 12 байт

Приводим к байтам для смещения

Индекс

!!! Ссылка на значение

Сколько байт копируем

```
// Получения элемента
printf("Value: %d\n", *(int*)((char*)arr + 3 * elem_size));
```

Получаем значение по указателю

Говорим компилятору что указатель на int

Для int и для индекса 3 нам нужно сместиться на 12 байт

Понеслось

```
typedef struct {
 void* data;        // Указатель на блок данных
 size_t elem_size;  // Размер элементов
 size_t size;       // Текущее количество элементов
 size_t capacity;   // Вместимость (выделенная память в элементах)
} Vector;
```

Размер элемента:

- int ⟶ sizeof(int) = 4 байта
- short int ⟶ sizeof(short int) = 2 байта
- double ⟶ sizeof(double) = 8 байта
- char ⟶ sizeof(char) = 1 байт
- char* ⟶ sizeof(???) = 100 байт (пока ограничиваем строки 100 байтами)
- struct Point ⟶ sizeof(struct Point) = тут всё по классике

Изменяем реализацию

ИТМО

```
Vector *createVector(size_t elem_size) {
    ...
    vector->data = malloc(MIN_SIZE *
elem_size);

    ...
    return vector;
}
```

```
int appendVectorItem(Vector *vector, void
*el) {
    ...

    memcpy((char *)vector->data +
vector->size * vector->elem_size, el,
vector->elem_size);

    ...
    return 0;
}
```

```
void *popVectorItem(Vector *vector,
size_t index)
{
     ...

    void *copy =
malloc(vector->elem size);
    if (!copy) return NULL;

    void *item = getVectorItem(vector,
index);
    memcpy(copy, item,
vector->elem_size);


     ...
}
```

Функция сглаживания снова

ИТМО

```
Vector* smoothV2(Vector* arr, const u_int8_t window_size) {
    if (window_size < 2) RETURN_ERROR("Wrong size", NULL);
    if (window_size >= arr->size) RETURN_ERROR("Wrong size", NULL);

    Vector* arr_out = createVector();
    if (arr_out == NULL) RETURN_ERROR("Error vector creation", NULL);

    int8_t window[window_size];
    for (int i = 0; i < arr->size - window_size + 1; i++) {

        for (int j = 0; j < window_size; j++)
            window[j] = getVectorItem(arr, i + j);

        int8_t value = meanValue(window, window_size);
        if (appendVectorItem(arr_out, &value) == -1) {
            vectorFree(arr_out);
            RETURN_ERROR("Не удалось релоцировать память", NULL);
        }
    }

    return arr_out;
}
```

Вот это еще бесит

Что можно сделать?

Связный список

ИТМО

```c
typedef struct Node
{
    void *data;
    struct Node *next;
} Node;
```

```c
typedef struct
{
    Node *head;
    size_t elem_size;
} GenericList;
```

```c
GenericList *createList(size_t elem_size);

void appendItem(GenericList *list, void *data);

unsigned int findItem(GenericList *list, void *value,
EqualsFunc cmp);

void *popItem(GenericList *list, size_t index);

void freeList(GenericList *list);

unsigned int listLength(GenericList *list);
```

По лямбда функциям в C

ИТМО

```c
typedef int (*EqualsFunc)(const void *a, const void *b);

static inline int intEquals(const void *a, const void *b){
    return *(int* )a == *(int* )b;
}

unsigned int findItem(GenericList *list, void *value, EqualsFunc cmp) {
    if (!list || !cmp || !list->head) return (unsigned int)-1;

    Node *current = list->head;
    unsigned int index = 0;

    while (current) {
        if (cmp(current->data, value)) break;

        current = current->next;
        index++;
    }

    return (unsigned int)-1; // Элемент не найден
}
```

```c
Vector* smoothV3(Vector* arr, const u_int8_t window_size) {
    if (window_size < 2) RETURN_ERROR("Validation error", NULL);
    if (window_size >= arr->size) RETURN_ERROR("Validation error", NULL);

    Vector* arr_out = createVector(arr->elem_size);
    if (arr_out == NULL) RETURN_ERROR("Ошибка выделения памяти под массив", NULL);

    GenericList* list = createList(arr->elem_size);
    for (int i = 0; i < arr->size; i++) {
        appendItem(list, getVectorItem(arr, i));

        if (i < window_size - 1) continue;
        else free((int8_t*) popItem(list, 0));

        int8_t value = meanListValue(list);
        if (appendVectorItem(arr_out, &value) == -1) {
            vectorFree(arr_out);
            RETURN_ERROR("Не удалось релоцировать память", NULL);
        }
    }

    return arr_out;
}
```



Мне кажется

Или у вас есть вопросы?

ITMO more than a UNIVERSITY

Ivan.Khodnenko@itmo.ru